AEGIS

Agentic Engine for Global Investigation & Safety

Every Signal Matters. Every Life Counts.

Platform: UiPath Hackathon 2026

Organization: hackathon26_047 · staging.uipath.com

Stack: TypeScript · React · Python · Supabase · Maestro BPMN

Folder: AI Human Trafficking Detection & Prevention Network / AEGIS

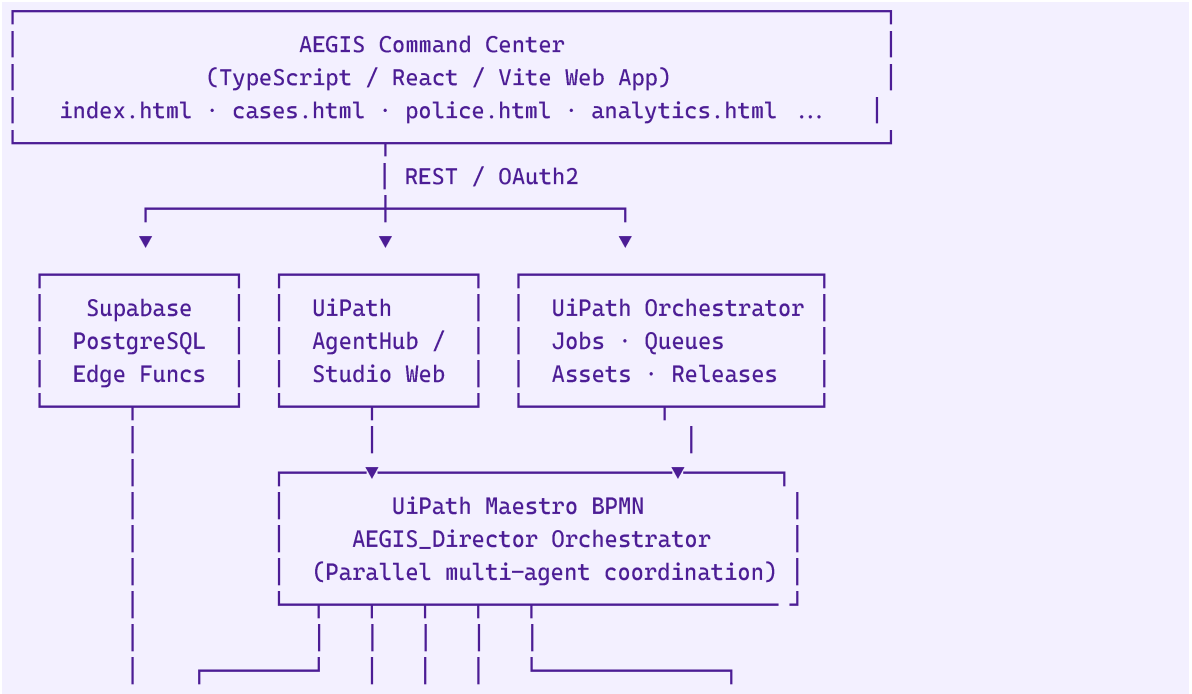
AEGIS — Agentic Engine for Global Investigation & Safety

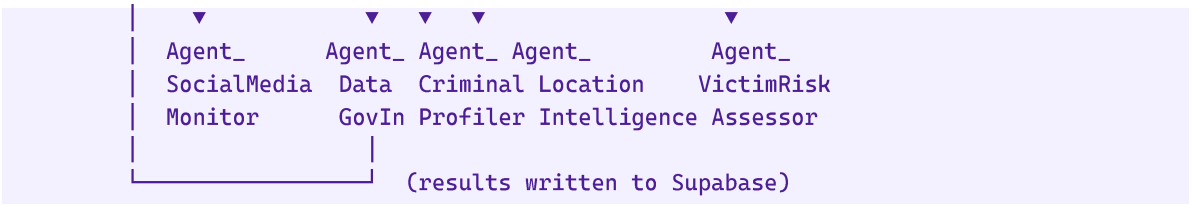
> ***Every Signal Matters. Every Life Counts.***

AEGIS is a multi-agent AI platform built on UiPath for detecting, investigating, and coordinating responses to human trafficking and missing persons cases. It combines a React/TypeScript command center web app, UiPath Python coded agents, Maestro BPMN orchestration, and a Supabase backend into a unified operational system.

Table of Contents

Architecture Overview





Tech Stack:

Layer	Technology
Frontend	TypeScript · React 19 · Vite 8 · Vanilla CSS
Backend DB	Supabase (PostgreSQL + Row Level Security)
Edge Functions	Deno / TypeScript (Supabase Edge)
Agent Runtime	UiPath Python SDK · LangGraph
Orchestration	UiPath Maestro BPMN · Orchestrator
Auth	UiPath OAuth2 PKCE (staging.uipath.com)
Web Intelligence	Firecrawl API
Deployment	UiPath Studio Web · Coded App (nupkg)

Web App — Pages & Features

The app is a multi-page TypeScript/React application deployed as a UiPath Coded App.

Pages

Page	File	Description
Command Center	index.html	Main dashboard — KPIs, agent network map, notifications, live feed
Authentication	auth.html	OAuth2 PKCE login & callback handler
Case Registry	cases.html	Central missing persons & trafficking case management
Case Detail	detail.html	Individual case deep-dive with evidence, timeline, agent findings
Analytics	analytics.html	Historical data analytics, predictive risk scoring
Document Scanner	scanner.html	Agentic DB scanner — matches criminal records against active cases
Police Portal	police.html	Case submission portal for field officers
Surveillance	surveillance.html	Live & recorded CCTV/video intelligence
World Monitor	worldmonitor.html	Global trafficking signal intelligence map

Settings	settings.html	User, org, and integration settings
Criminals DB	criminals.html	Criminal records database with case matching

Key Frontend Modules

File	Purpose
public/styles.css	Global design system (CSS variables, dark/light, responsive)
public/config.js	Runtime configuration (UiPath URLs, Supabase keys, org settings)
public/script.js	Shared utilities (auth helpers, API wrappers, UI components)
public/supabase-client.js	Supabase JS client initialization
public/aegis-flow-connector.js	UiPath Maestro Flow trigger integration
public/agent-page.js	Agent execution UI (terminal output, streaming)
src/uipath-chatbot.ts	UiPath Conversational Agent integration
src/uipath-orchestrator.ts	Orchestrator API wrapper (jobs, queues, assets)

Python Agents

All agents are in the `agents/` directory and follow the UiPath Python SDK pattern with `Input`/`Output` Pydantic models.

Agent: search_agent.py

Function: `search_person(input: Input) -> Output`

Searches the missing persons database for matching records based on name, description, location, and other identifiers.

```python
class Input(BaseModel):
    query: str          # Search query (name, description, etc.)
    filters: dict       # Optional filters (location, age range, etc.)

class Output(BaseModel):
    matches: list       # List of matching persons with confidence scores
    total: int          # Total matches found
```

Agent: document_analyser.py

Function: `analyse_document(input: Input) -> Output`

Performs Named Entity Recognition (NER) and information extraction on documents — police reports, witness statements, evidence files.

```python
class Input(BaseModel):
    document_text: str  # Raw text or extracted text from document
    document_type: str  # e.g. "police_report", "witness_statement"

class Output(BaseModel):
    entities: dict      # Extracted: names, locations, dates, organizations
    facts: list         # Key factual statements
    confidence: float   # Overall extraction confidence
```

Agent: orchestrator_manager.py

Function: `manage_orchestrator(input: Input) -> Output`

Controls UiPath Orchestrator programmatically — triggers jobs, queries queues, lists available robots, and monitors process execution.

```python
class Input(BaseModel):
    action: str         # "trigger_job" | "query_queue" | "list_robots"
    process_name: str   # Target process name
    input_args: dict    # Arguments to pass to the process

class Output(BaseModel):
    job_key: str        # Started job key
    status: str         # Job status
    result: dict        # Job output/result data
```

Agent: surveillance_agent.py

Function: `main(input: Input) -> Output`

CCTV and video intelligence analysis. Processes video feeds to detect persons of interest, track movement, and generate intelligence reports.

Agent: web_intelligence_agent.py

Web scraping and open-source intelligence (OSINT) gathering using Firecrawl. Monitors dark web signals, social media, and public sources for trafficking indicators.

Uses `firecrawl_client.py` as the API wrapper.

Running Agents Locally

```bash
# Install Python dependencies
```

```
pip install uipath

# Initialize agent project
uv run uipath init

# Run an agent with test input
uv run uipath run agents/search_agent.py '{"query": "Jane Doe", "filters": {}}'

# Run with input file
uv run uipath run agents/document_analyser.py --file test_input.json

# Evaluate agent performance
uv run uipath eval
```

UiPath Processes & Orchestration

Maestro BPMN: AEGIS_Director

The primary orchestration flow. Triggered when a new case is registered.

Flow:

```
Start → Agent_AEGIS_Director (analysis & routing)
      → [Parallel Gateway]
            ├── Agent_AEGIS_SocialMediaMonitor
            ├── Agent_DataDotGovIn
            ├── Agent_AEGIS_CriminalProfiler
            ├── Agent_AEGIS_LocationIntelligence
            └── Agent_AEGIS_VictimRiskAssessor
      → [Join Gateway]
      → Agent_AEGIS_SummarizerAndReporter
      → End
```

Folder: `AI Human Trafficking Detection & Prevention Network/AEGIS`

Process Key: `AEGIS_Director.agentic.AEGIS_Director`

Triggering via API

The app triggers the Director flow via a Supabase Edge Function:

TYPESCRIPT

```
// supabase/functions/start-bpmn/index.ts
POST https://<supabase-project>.supabase.co/functions/v1/start-bpmn
Authorization: Bearer <supabase-anon-key>
Content-Type: application/json

{
  "case_id": "CASE-001",
  "case_data": { ... },
  "folder_path": "AI Human Trafficking Detection & Prevention Network/AEGIS"
}
```

The edge function calls Orchestrator to start the `AEGIS_Director` process:

```
POST /odata/Jobs/UiPath.Server.Configuration.OData.StartJobs
```

Maestro Case: AEGIS_MaestroCase

Long-running case management workflow with human-in-the-loop (HITL) stages for reviewing agent findings, validating evidence, and authorizing actions.

Supabase Backend

Project Ref: `mtbfgbyzgqenkzfejbvr`

Region: ap-south-1 (Mumbai — for India deployment)

Database Schema

cases Table

Stores active missing persons and trafficking cases.

```sql
cases (
  id uuid PRIMARY KEY,
  case_id text UNIQUE,           -- Human-readable case ID (CASE-001)
  officer_name text,
  station_name text,
  victim_name text,
  victim_age int,
  last_seen_location text,
  description text,
  status text,                   -- 'pending' | 'active' | 'resolved' | 'closed'
  created_at timestamptz,
  updated_at timestamptz
)
```

criminals Table

Criminal records database for cross-referencing against cases.

```sql
criminals (
  id uuid PRIMARY KEY,
  name text,
  aliases text[],
  known_locations text[],
  crime_types text[],
  risk_level text,               -- 'low' | 'medium' | 'high' | 'critical'
  photo_url text,
  created_at timestamptz
)
```

aegis_case_results Table

Agent findings written back after Maestro flow completes.

```sql
aegis_case_results (
  id uuid PRIMARY KEY,
  case_id text REFERENCES cases(case_id),
  agent_name text,
  result_type text,
  result_data jsonb,
  confidence float,
  created_at timestamptz
)
```

Edge Functions

Function	Trigger	Purpose
start-bpmn	HTTP POST	Start AEGIS_Director Maestro flow for a case
trigger-bpmn	HTTP POST	Trigger other BPMN flows
invoke-datagov-agent	HTTP POST	Invoke the DataDotGovIn agent
poll-datagov-job	HTTP POST	Poll job status for DataGov agent runs
case-results-webhook	HTTP POST	Receive completed Maestro job results

Deploy Edge Functions

```bash
supabase functions deploy start-bpmn --project-ref mtbfgbyzgqenkzfejbvr
supabase functions deploy trigger-bpmn --project-ref mtbfgbyzgqenkzfejbvr
supabase functions deploy invoke-datagov-agent --project-ref mtbfgbyzgqenkzfejbvr
supabase functions deploy poll-datagov-job --project-ref mtbfgbyzgqenkzfejbvr
supabase functions deploy case-results-webhook --project-ref mtbfgbyzgqenkzfejbvr
```

Database Setup

```bash
# Apply full schema
supabase db push --db-url <connection-string>

# Or run individual migrations
psql -f supabase/cases-schema.sql
psql -f supabase/criminals-schema.sql
psql -f supabase/settings-schema.sql
psql -f supabase/20260624_police_full_setup.sql
psql -f supabase/20260625_aegis_case_results.sql
```

External APIs

UiPath Platform (staging.uipath.com)

API	Purpose
POST /odata/Jobs/UiPath.Server.Configuration.OData.StartJobs	Trigger automation jobs
GET /odata/Jobs	List/monitor running jobs
POST /odata/Processes/UiPath.Server.Configuration.OData.UploadPackage()	Upload coded app packages
GET /odata/Assets	Retrieve configuration assets
POST /apps_/default/api/v1/default/models/apps/codedapp/publish	Register coded app

Base URL: `https://staging.uipath.com/hackathon26_047/DefaultTenant`

Auth: Bearer token (OAuth2 PKCE — 1h expiry)

Firecrawl

Used by `web_intelligence_agent.py` for web scraping and OSINT.

```python
# Usage
from firecrawl_client import FirecrawlClient
client = FirecrawlClient(api_key=FIRECRAWL_API_KEY)
result = client.scrape(url="https://...", formats=["markdown"])
```

Required: `FIRECRAWL_API_KEY` in `.env`

Supabase

JavaScript client initialized in `public/supabase-client.js`:

```javascript
const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY)
```

Required: `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` in `.env`

Environment Variables

Copy `.env.example` to `.env` and fill in all values.

UiPath Platform

```env
# Base URL for UiPath tenant
UIPATH_URL=https://staging.uipath.com/hackathon26_047/DefaultTenant

# Organization & Tenant UUIDs (from UiPath Cloud portal)
UIPATH_ORGANIZATION_ID=e1d20aec-58e2-4855-8fef-32f3d211e75e
UIPATH_TENANT_ID=39e1c6c7-9d5e-49ef-802e-32d2824fbff6
```

```
# OAuth2 Client (for machine-to-machine flows)
UIPATH_CLIENT_ID=<your-client-id>
UIPATH_CLIENT_SECRET=<your-client-secret>
UIPATH_SCOPE=OR.Execution OR.Folders OR.Jobs

# CLI deploy token (1h TTL — obtained via 'npm run login:uip')
UIPATH_CLI_TOKEN=<pkce-token>

# Orchestrator user token (for browser runtime)
UIPATH_ACCESS_TOKEN=<user-access-token>
VITE_UIPATH_ACCESS_TOKEN=<same-token-for-vite>
UIPATH_USER_TOKEN=<orchestrator-jwt>

# CLI OAuth2 client (for npm run login:uip)
UIPATH_CLI_CLIENT_ID=89fc94c0-6c49-436d-806c-996d7f6ee2e5
UIPATH_CLI_CLIENT_SECRET=<secret>
```

Orchestrator References

```ENV
# AEGIS Director process
UIPATH_PROCESS_KEY=AEGIS_Director.agentic.AEGIS_Director
UIPATH_RELEASE_KEY=9f617f16-9aee-4de1-ad67-a8b104c21b51

# Folder where AEGIS agents are deployed
UIPATH_FOLDER_PATH=AI Human Trafficking Detection & Prevention Network/AEGIS_Director
UIPATH_FOLDER_ID=3146136
UIPATH_FOLDER_KEY=09abb24a-d60a-438a-98f3-67accb78fb78
```

Supabase

```ENV
VITE_SUPABASE_URL=https://mtbfgbyzgqenkzfejbvr.supabase.co
VITE_SUPABASE_ANON_KEY=<anon-key>
SUPABASE_SERVICE_ROLE_KEY=<service-role-key>
```

External Services

```ENV
FIRECRAWL_API_KEY=<your-firecrawl-api-key>
```

Local Development

Prerequisites

- Node.js 20+
- Python 3.11+
- `uv` package manager (`pip install uv`)
- Supabase CLI (`npm install -g supabase`)

- UiPath CLI (`npm install -g @uipath/uipath-cli`)

Setup

```bash
# 1. Clone the repository
git clone <repo-url>
cd uipath-ts-demo

# 2. Install Node dependencies
npm install

# 3. Install Python dependencies
uv sync

# 4. Copy and fill environment variables
cp .env.example .env
# Edit .env with your credentials

# 5. Start the development server
npm run dev
```

The dev server runs at `http://localhost:5173`

Available npm Scripts

Script	Command	Description
dev	vite	Start local dev server
build	tsc -b && vite build	Production build to dist/
login	scripts/login.ps1	Interactive OAuth login
login:uip	scripts/login-uip.ps1	PKCE login for CLI deploy token
login:status	scripts/login-status.ps1	Check token expiry
ts:pack	scripts/ts-pack.ps1	Build & package as nupkg
ts:publish	scripts/ts-publish.ps1	Upload nupkg to Orchestrator
ts:deploy	scripts/ts-deploy.ps1	Deploy via CLI
push	scripts/codedapp-push.ps1	Full push to Studio Web
py:pack	uipath pack	Package Python agents
py:publish	scripts/python-publish.ps1	Publish Python agents
py:deploy	scripts/python-deploy.ps1	Deploy Python agents

Deployment

Web App Deployment

The app deploys as a UiPath Coded App (TypeScript webapp nupkg) to Studio Web.

Step 1 — Authenticate

```bash
npm run login:uip
# Opens browser for PKCE OAuth flow
# Token stored in .env as UIPATH_CLI_TOKEN (1h expiry)
```

Step 2 — Pack

```bash
npm run ts:pack
# Prompts for version (e.g. 1.0.2)
# Builds with Vite, copies public/ assets, packs to .uipath/aegis-app-webapp.1.0.2.nupkg
```

Step 3 — Publish to Orchestrator

```bash
npm run ts:publish
# Uploads nupkg to Orchestrator package feed
```

Step 4 — Deploy via Studio Web

12. Go to UiPath Studio Web
13. Navigate to folder `AI Human Trafficking Detection & Prevention Network/AEGIS`
14. Open the `aegis-app-webapp` app
15. Select **Deploy** → choose version `1.0.2`
16. App goes live at: `https://hackathon26_047.staging.uipath.host/<path-name>/`

> ***Note:*** *CLI deploy (`npm run ts:deploy`) is currently blocked on the staging tenant due to a disabled NuGet v2 feed. Use Studio Web UI for the final deploy step.*

Python Agents Deployment

```bash
# Pack the Python agent project
npm run py:pack

# Publish to Orchestrator
npm run py:publish

# Deploy
npm run py:deploy
```

Supabase Edge Functions

```bash
# Deploy all edge functions
supabase functions deploy --project-ref mtbfgbyzgqenkzfejbvr

# Deploy individual function
supabase functions deploy start-bpmn --project-ref mtbfgbyzgqenkzfejbvr
```

Authentication Flow

The web app uses UiPath OAuth2 PKCE for authentication.

```
User visits index.html
    |
    ├── Session found in sessionStorage? ──Yes──▶ Show dashboard
    |
    └── No session
            |
            ▼
        Redirect to auth.html
            |
            ▼
        Auth guard checks URL params
            |
            ├── Has ?code=&state= (OAuth callback)? ──▶ Exchange code for token
            |                                                    |
            |                                                    ▼
            |                                          Store in sessionStorage
            |                                                    |
            |                                                    ▼
            |                                          Redirect to index.html
            |
            └── No code → Redirect to UiPath OAuth2 authorize endpoint
```

OAuth2 Config (in `index.html` meta tags):

```HTML
<meta name="uipath:client-id" content="e8a47047-3001-4212-9a1d-cebceaf72e26">
<meta name="uipath:scope" content="OR.Execution OR.Folders OR.Jobs ConversationalAgents
Traces.Api">
<meta name="uipath:org-name" content="hackathon26_047">
<meta name="uipath:tenant-name" content="DefaultTenant">
```

Folder Structure

```
uipath-ts-demo/
├── agents/                    # Python coded agents
|   ├── search_agent.py        # Missing persons search
|   ├── document_analyser.py   # NER & document extraction
|   ├── orchestrator_manager.py # Orchestrator job control
|   ├── surveillance_agent.py  # CCTV intelligence
|   ├── web_intelligence_agent.py # OSINT / web scraping
|   └── firecrawl_client.py    # Firecrawl API wrapper
├── public/                    # Static web assets
|   ├── index.html             # Command center dashboard
|   ├── auth.html              # OAuth2 login handler
|   ├── cases.html             # Case registry
|   ├── police.html            # Police submission portal
```

```
│   ├── analytics.html          # Analytics dashboard
│   ├── surveillance.html       # CCTV monitoring
│   ├── worldmonitor.html       # Global intelligence map
│   ├── scanner.html            # Criminal DB scanner
│   ├── criminals.html          # Criminal records DB
│   ├── detail.html             # Case detail view
│   ├── settings.html           # Settings
│   ├── styles.css              # Global design system
│   ├── config.js               # Runtime configuration
│   ├── script.js               # Shared utilities
│   ├── supabase-client.js      # Supabase client init
│   └── aegis-flow-connector.js # Flow trigger integration
├── src/                        # React/TypeScript source
│   ├── main.tsx
│   ├── App.tsx
│   ├── uipath-chatbot.ts
│   └── uipath-orchestrator.ts
├── supabase/                   # Supabase backend
│   ├── functions/              # Edge Functions (Deno)
│   │   ├── start-bpmn/
│   │   ├── trigger-bpmn/
│   │   ├── invoke-datagov-agent/
│   │   ├── poll-datagov-job/
│   │   └── case-results-webhook/
│   ├── schema.sql              # Full database schema
│   ├── cases-schema.sql
│   ├── criminals-schema.sql
│   └── migrations/             # DB migration files
├── scripts/                    # PowerShell deployment scripts
│   ├── login-uip.ps1           # PKCE auth login
│   ├── ts-pack.ps1             # Build & package nupkg
│   ├── ts-publish.ps1          # Upload to Orchestrator
│   ├── ts-deploy.ps1           # CLI deploy
│   ├── codedapp-push.ps1       # Full Studio Web push
│   ├── python-publish.ps1      # Python agent publish
│   └── python-deploy.ps1       # Python agent deploy
├── docs/                       # Documentation
│   ├── uipath-auth-guide.md
│   └── BEGINNER_GUIDE.md
├── .agent/                     # UiPath agent SDK reference docs
├── package.json                # Node project config (name: aegis-app)
├── pyproject.toml              # Python project config
├── vite.config.ts              # Vite build config (multi-page)
├── uipath.json                 # UiPath OAuth & project config
└── .env                        # Environment variables (do not commit)
```

Security Notes

- Never commit `.env` — it contains tokens and secrets
- `.uipath/.auth.json` contains cached OAuth tokens — gitignored
- `supabase/.temp/` contains local Supabase state — gitignored

- All Supabase tables use Row Level Security (RLS)
- UiPath tokens expire after 1 hour — run `npm run login:uip` to refresh before deploying

---

Built For

UiPath Hackathon 2026

Organization: `hackathon26_047` (staging.uipath.com)

Team: AEGIS Guardian

Full name: **Agentic Engine for Global Investigation & Safety**